

CICLO DE COMPLEMENTACIÓN

LICENCIATURA EN CIENCIA DE DATOS

CLCD 182 - SEMINARIO DE
ACTUALIZACIÓN I

Clases

Semana 05





Temas a Desarrollar

- Operadores aritméticos
- Operadores relacionales
- Operadores lógicos
- Operadores bitwise
- Operadores identidad
- Operadores membresía
- Operadores asignación
- Operadores warlus



Operadores aritméticos

A continuación, condensamos en la siguiente tabla todos ellos con un ejemplo, donde `x=10` y `y=3`.

Operador	Nombre	Ejemplo
+	Suma	$x + y = 13$
-	Resta	$x - y = 7$
*	Multiplicación	$x * y = 30$
/	División	$x/y = 3.333$
%	Módulo	$x\%y = 1$
**	Exponente	$x ** y = 1000$
//	Cociente	3



Operadores aritméticos

Particularidades del operador suma

```
La suma de dos str "2" + "2" es: 22  
La suma de dos listas [1, 3] + [6, 7] es [1, 3, 6, 7]
```

Particularidades del operador de multiplicación

```
[ ] print("Hola" * 3) #HolaHolaHola
```



```
HolaHolaHola
```



Operadores aritméticos

Particularidades del módulo o resto

```
La operación 7%3 da un resultado de 1 | [en otros lenguajes daría 1]  
La operación -7%3 da un resultado de 2 | [en otros lenguajes daría -1]  
La operación 7%-3 da un resultado de -2 | [en otros lenguajes daría 1]  
La operación -7%-3 da un resultado de -1 | [en otros lenguajes daría -1]
```



Operadores aritméticos



Particularidades

de la división

entera.

```
a // b = 7 // 3 = 2
Si lo hacemos por pasos 2.3333 -> 2
```

Nuevos valores de a y b

```
a // b = -7 // 3 = -3
Si lo hacemos por pasos -2.3333 -> -3
```

Notar que el resultado es -2.3333 y como lo lleva a 3 cuando otros lenguajes van a mostrar -2. Esto es una particularidad única de Python.

Nuevos valores de a y b

```
a // b = 7 // -3 = -3
Si lo hacemos por pasos -2.3333 -> -3
```

Notar que el resultado es -2.3333 y como lo lleva a 3 cuando otros lenguajes van a mostrar -2. Esto es una particularidad única de Python.

Nuevos valores de a y b

```
a // b = -7 // -3 = 2
Si lo hacemos por pasos 2.3333 -> 2
```



Operadores aritméticos



Orden de los operadores

- `()` Paréntesis
- `**` Exponente
- `-x` Negación
- `*`, `/`, `//`, `%`
- `+`, `-`

Operadores relacionales



Operador	Nombre	Ejemplo	Resultado
==	Igual	x == y	False
!=	Distinto	x != y	True
>	Mayor	x > y	False
<	Menor	x < y	True
>=	Mayor o igual	x >= y	False
<=	Menor o igual	x <= y	True



Operadores relacionales

Operador ==

```
La comparación 4==4 es True
La comparación 4==5 es False
La comparación 4==4.0 es True
La comparación 0==False es True
La comparación "asd"=="asd" es True
La comparación "asd"=="asdf" es False
La comparación 2=="2" es False
La comparación [1, 2, 3] == [1, 2, 3] es True
```

Operador !=

```
La comparación 4!=4 es False
La comparación 4!=5 es True
La comparación 4!=4.0 es False
La comparación 0!=False es False
La comparación "asd"!="asd" es False
La comparación "asd"!="asdf" es True
La comparación 2!="2" es True
La comparación [1, 2, 3] != [1, 2, 3] es False
```



Operadores relacionales

Operadores

> o < o >= o <=

```
x > y -> 2 > 3 = False
y > x -> 3 > 2 = True
```

```
La comparación [1, 2] > [10, 10] es False
La comparación [10, 10] > [1, 2] es True
```

```
Que pasa si las listas tienen diferente tamaño.
La comparación [10, 10, 11] > [1, 2] es True
La comparación [10, 10] > [1, 2, 22] es True
```

```
Que pasa si las listas tienen diferente tamaño.
La comparación [1, 2] > [10, 10, 11] es False
La comparación [1, 2, 22] > [10, 10] es False
```

True es igual a 1

```
El valor True==1 es True
El valor True>0.999 es True
```

Solo compara las posiciones que coinciden



Operadores lógicos



not

variable_1	Operación	Resultado
True	not True	False
False	not False	True

and

variable_1	variable_2	Operación	Resultado
True	True	True and True	True
True	False	True and False	False
False	False	False and False	False
False	True	False and True	False

or

variable_1	variable_2	Operación	Resultado
True	True	True or True	True
True	False	True or False	True
False	False	False or False	False
False	True	False or True	True



Operadores bitwise

Ampersand (and)

variable_1	variable_2	Operación	Resultado
1	1	1 & 1	1
1	0	1 & 0	0
0	0	0 & 0	0
0	1	0 & 1	0

Pipe (or)

variable_1	variable_2	Operación	Resultado
1	1	1 1	1
1	0	1 0	1
0	0	0 0	0
0	1	0 1	1

Tilde (not)

~c sería -(c + 1)

variable_1	Operación	Resultado
0	~0	1 (negativo)
1	~1	10 (negativo)



Operadores bitwise

Acento circunflejo (xor)

variable_1	variable_2	Operación	Resultado
1	1	$1 \wedge 1$	0
1	0	$1 \wedge 0$	1
0	0	$0 \wedge 0$	0
0	1	$0 \wedge 1$	1

Doble mayor

```
d=0b1000
print(f'El valor {d:b} en entero es {int(d)}')
print(f'La operación binaria {d>>2:b}>>2 es {d>>2}')
```

El valor 1000 en entero es 8
La operación binaria 10>>2 es 2

Doble menor

```
d=0b0001
print(f'El valor {d:b} en entero es {int(d)}')
print(f'La operación binaria {d<<2:b}<<2 es {d<<2}')
```

El valor 1 en entero es 1
La operación binaria 100<<2 es 4



Operadores Identidad

Operador	Nombre
is	Devuelve True si hacen referencia a el mismo objeto
is not	Devuelve False si no hacen referencia a el mismo objeto

```
a = 10
b = 10

print(f'El valor de a es {a} y su id es {id(a)}')
print(f'El valor de b es {b} y su id es {id(b)}')
print(f'La operación a is b es {a is b}')
```

El valor de a es 10 y su id es 133730794144272
El valor de b es 10 y su id es 133730794144272
La operación a is b es True

Operadores de membresía



Operador	Nombre	Ejemplo
in	<code>True</code> si el elemento esta contenido	<code>xxx</code>
not in	<code>False</code> si el elemento no esta contenido	<code>xxx</code>

```
a=3
b=[1, 2, 3]
print(f'El valor {a} in {b}: {a in b}')
```

```
El valor 3 in [1, 2, 3]: True
```



Operadores de asignación

Operadores aritméticos.

Operador	Ejemplo	Equivalente
----------	---------	-------------

=	x=7	x=7
---	-----	-----

+=	x+=2	x=x+2 = 9
----	------	-----------

-=	x-=2	x=x-2 = 5
----	------	-----------

=	x=2	x=x*2 = 14
----	------	------------

/=	x/=2	x=x/2 = 3.5
----	------	-------------

%=	x%=2	x=x%2 = 1
----	------	-----------

//=	x//=2	x=x//2 = 3
-----	-------	------------

=	x=2	x=x**2 = 49
-----	-------	-------------

Operadores binarios.

Para cada caso `x=7`

La representación de los números binarios es:

- 28 = 11100
- 7 = 1111
- 5 = 0111
- 2 = 0010
- 1 = 0001

Operador	Ejemplo	Equivalente
----------	---------	-------------

&=	x&=2	x=x&2 = 2
----	------	-----------

=	x =2	x=x 2 = 7
---	------	-----------

^=	x^=2	x=x^2 = 5
----	------	-----------

>>=	x>>=2	x=x>>2 = 1
-----	-------	------------

<<=	x<<=2	x=x<<2 = 28
-----	-------	-------------



Operador warlus

- Se introdujo en python 3.8
- Se utiliza con el operador `:=`
- Asigna un valor y lo retorna

Ejemplo:

```
file = '/content/test.txt'
with open(file) as f:
    line = f.readline()
    while line:
        print(line, end='')
        line = f.readline()
```



```
with open(file) as f:
    while line := f.readline():
        print(line, end='')
```